# CHOZ

**User Guide**

Version 1.3.10 · Linux (ALSA / JACK / PipeWire)

**About this document.** This guide documents choz 1.3.10. It is written for the terminal application; choz's effects and its two note generators are also published as a CLAP plugin for other hosts, which behaves as described in chapter 6 but is driven by the host's own interface.
Screenshots were taken from choz running on Linux with the JACK/PipeWire backend. Device, bank and preset names in them belong to that installation and will differ on yours.

# Contents

# 1. Welcome to choz

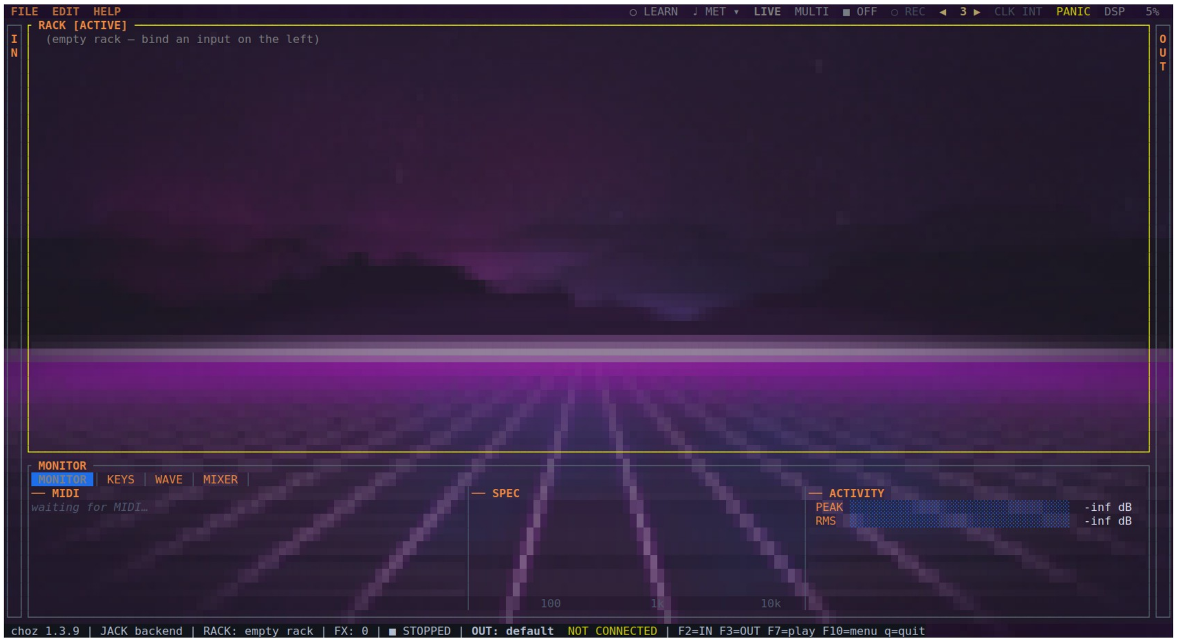


*Figure 1.1 — choz just opened: the menu bar, an empty RACK, the IN and OUT drawers at the edges, and the MONITOR panel along the bottom.*

## 1.1 What choz is

choz is an audio plugin host for the terminal. It gives you a rack of tabs; each tab takes notes or audio in, loads an instrument, chains effects onto it, and leaves either through the main mix or through a port of its own on the JACK/PipeWire graph.

- **Instruments:** SoundFont (SF2), SFZ, WAV, and CLAP, LV2, VST2, VST3, DSSI and LADSPA plugins, plus Pure Data patches.
- **Effects:** 56 of choz's own, plus every effect plugin installed on the machine, up to twelve per tab.
- **Note sources:** any MIDI port, choz's own ALSA and JACK ports, OSC, the computer keyboard, an arpeggiator and a step sequencer per tab, and audio-to-note tracking from a microphone.
- **Outputs:** the main stereo mix, four groups, and one direct out per tab for recording each tab on its own track in a DAW.

## 1.2 About this guide

Chapters 1 to 3 are the introduction: what choz needs, how to install it, and what is on the screen. Chapter 2 is a short walk-through that gets a sound out of it. Chapters 4 to 13 are the reference, one chapter per area of the program. Chapters 14 and 15 are lookup tables — shortcuts, environment variables, files. Chapter 16 collects the problems people actually hit, and chapter 17 is a glossary of the terms used throughout.

## 1.3 Document conventions

| Convention | Meaning |
|---|---|
| **BOLD CAPITALS** | Something written on the screen: a button, a panel, a knob — **MIXER**, **PANIC**, **SENS**. |
| `Monospace` | Something you type, or a name outside choz: a command, a file, a port — `choz:midi_in`. |
| F2, Enter, `a` | A key to press. Capitals mean the shifted key: `A` is Shift+a. |
| → | A path through menus: EDIT → Settings → THEME. |

**Note.** Grey boxes like this one carry something worth knowing before it bites: a limit, an interaction with another program, or a setting that only takes effect on the next start.

## 1.4 System requirements

| Requirement | Detail |
|---|---|
| System | Linux with glibc; x86-64, aarch64 or armv7 |
| Audio | `libasound.so.2` (required); `libjack.so.0` (optional, opened at runtime) |
| Terminal | 24-bit colour, at least 80×24 — 120×40 to work comfortably. kitty, Ghostty and WezTerm also draw the wallpaper through the terminal's graphics protocol |
| Plugin windows | X11 or XWayland, and only to open a plugin's own window: every plugin parameter also appears as a knob in the rack |
| Privileges | `rtprio` and `memlock` (membership of the `audio` group) recommended for small buffers |

## 1.5 Installing

```
# from a release (no toolchain needed)
tar xzf choz-1.3.10-x86_64-unknown-linux-gnu.tar.gz
cd choz-1.3.10-x86_64-unknown-linux-gnu
./install.sh

# from the repository
cargo build --release
./packaging/install.sh --binary target/release/choz
```

There are also `.deb` and `.rpm` packages, and an Arch `PKGBUILD`. All of them add a desktop entry under Audio, so choz can be started from the application menu as well as from a shell.

The install leaves three things besides the binary: `choz.clap` (choz's 56 effects plus the arpeggiator and the step sequencer, as a CLAP plugin for other hosts), the wallpapers, and — when libpd is present — the `choz-pd-host` binary that runs Pure Data patches.

## 1.6 What an uninstall leaves behind

No uninstaller touches `~/.local/state/choz`. Projects, plugin paths, scan caches and interface preferences live there and survive upgrades and removals; see section 15 for what each file holds.

## 1.7 Starting choz

```
choz                        # an empty rack
choz project.yml            # open a project
choz instrument.sf2         # load a file straight into a tab
choz --osc-port 9000        # pin the OSC port instead of letting the OS pick
choz --version              # print the version and exit
choz --help                 # print the options and exit
```

## 1.8 Getting help

HELP → About choz shows the version and the build. The log at `~/.local/state/choz/choz.log` is the first place to look when something goes wrong, and chapter 16 explains the messages that are worth acting on. Running choz from a terminal also prints its diagnostics to standard error as they happen.

# 2. First steps

This chapter gets a sound out of choz and back into a DAW. It assumes nothing has been configured.

## 2.1 Play a SoundFont from a MIDI keyboard

1. Start choz. The rack is empty and says so.
2. Press **F2** to open the **IN** drawer. The note inputs are at the top: every MIDI port on the system, choz's own ports, and OSC.
3. Move to your keyboard's port and press **Enter**. choz makes tab 1 and binds the port to it — the row says → **tab 1**.
4. Press **F2** again to close the drawer, then `1` to open the source picker.
5. Type part of a SoundFont's name, or open the **SF2** tab, and press **Enter**. The bank loads into the tab.
6. Play. If nothing sounds, check the **MONITOR** panel at the bottom: it shows every MIDI message as it arrives, so it answers "is the keyboard reaching choz?" before you look anywhere else.

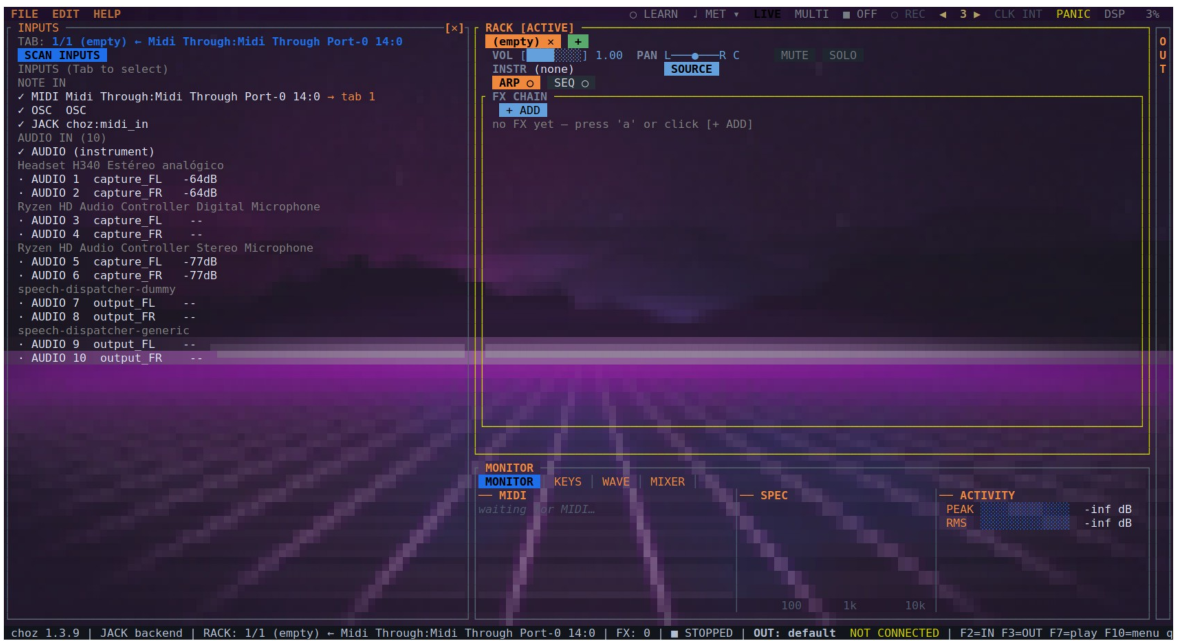


*Figure 2.1 — Accepting the MIDI row: choz makes tab 1 and feeds it from that port.*

## 2.2 Add an effect

1. With the tab active, press `a` to open the effect catalogue.
2. Pick a category on the left — **REVERB**, say — and an effect on the right. **BUILT-IN** at the top narrows the list to choz's own.
3. Press **Enter**. The effect is added to the tab's chain with all of its knobs.
4. Move between knobs with ↑ ↓, change one with the wheel or `w`/`s`, and switch the whole effect on or off with **Space**.

## 2.3 Play choz from a DAW and record it back

The usual round trip is: the DAW's MIDI out plays a choz tab, and choz's audio comes back into the DAW to be recorded.

1. **MIDI in.** In your DAW's MIDI device list, pick `choz MIDI IN` as the track's output. choz publishes that ALSA port itself, so nothing else is needed — see section 4.2.
2. **Audio out.** In choz, open EDIT → Settings → **AUDIO** and set the backend to **JACK**, then restart choz. It now has `choz:out_1..N` on the graph.
3. **Patch.** With any patchbay (qpwgraph, Carla, `jack_connect`), wire `choz:out_1` and `choz:out_2` into the DAW's inputs, and arm a track on them.
4. **Per-tab tracks.** To record several tabs separately, give each one its own direct out (section 4.6) and patch those instead of the main mix.

**Note.** On PipeWire, its JACK layer is enough — there is no need to start a separate `jackd`. The MIDI half of this round trip does not need the graph at all: `choz MIDI IN` is an ALSA sequencer port.

## 2.4 Save the project

Press **F10**, open **FILE** and choose **Save as**. A project is a readable YAML file that carries the rack, the routing, the effect chains, and the plugins' own state — the patch you chose in a plugin's browser comes back with the project.

# 3. choz overview

## 3.1 The screen

The screen has four fixed areas: the menu bar along the top, the RACK in the middle, the IN and OUT drawers at the left and right edges, and the MONITOR / KEYS / WAVE / MIXER panel along the bottom. The last line is the status line.

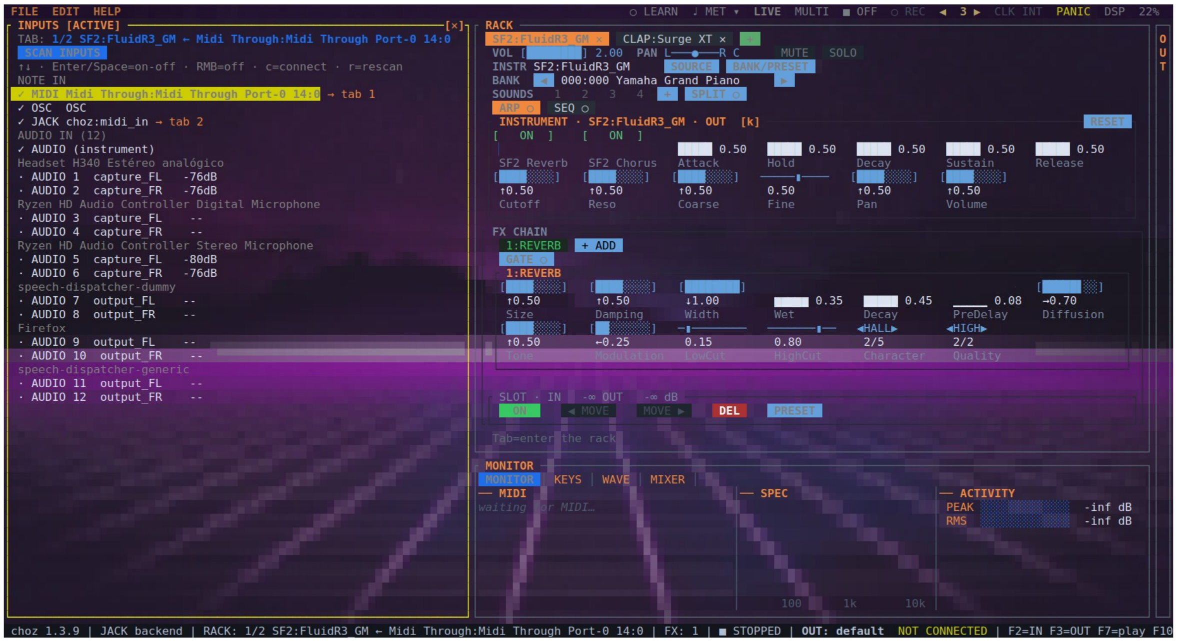


*Figure 3.1 — The whole screen: the IN drawer open on the left, tab 1 with a SoundFont and an internal reverb, and the bottom panel below.*

## 3.2 The menu bar and the transport cluster

FILE, EDIT and HELP sit on the left. Everything true of the whole rack rather than of one tab sits on the right:

| Control | What it does |
|---|---|
| **LEARN** | Arms MIDI learn by pointing: click a knob, then move a controller (section 8.3). |
| **TAP** | Plays the tempo in. Four clicks set it, and the fourth starts the metronome (section 10.3). |
| **MET and ▾** | The metronome and its options — click, tempo, time signature, grouping, sound, output, level. |
| **LIVE / MULTI** | Whether every tab plays at once or each listens on its own MIDI channel (section 8.1). |
| **▶️ ON / ■ OFF** | The transport: what starts every sequencer, every synced delay and the automation. |
| **REC** | Arms automation recording. A right-click clears the lanes. |
| **◀️ n ▶️** | The automation loop, in bars. Click the left half to shorten it, the right half to lengthen it. |
| **CLK** | The clock source: choz's own, or the external device driving it, by name. |
| **PANIC** | Silences every note that is still sounding. |
| **DSP** | What the audio thread is spending, as a percentage of the block budget. |

When the window is too narrow for the whole cluster, controls are given up in a fixed order: the ones a key can still reach go first, and the switches with no other home go last.

## 3.3 The rack

One tab per channel. A tab holds its input binding, its instrument, its effect chain, its arpeggiator and sequencer, its volume, pan and routing. `[` and `]` move between tabs; the mouse works everywhere.

## 3.4 The drawers

**F2** opens IN, **F3** opens OUT. They are lists of everything choz can hear and everything it can reach, and binding is done from them (chapter 4).

## 3.5 The bottom panel

**F5** cycles MONITOR, KEYS, WAVE and MIXER; clicking a tab goes straight there (section 11.4).

## 3.6 The status line

Version, audio backend, active tab, how many effects it carries, the transport, and where the output is going.

## 3.7 Mouse, wheel and keyboard

Everything on the screen is clickable, and the wheel over a knob changes its value. The keyboard reaches everything the mouse does: chapter 14 is the full list. Inside the rack, `k` moves the cursor between the instrument and the effects, the arrows walk the knobs, and **Enter** opens a list for a knob that has named values instead of a range.

# 4. Inputs and outputs

## 4.1 The IN drawer

At the top are the note inputs: every MIDI port on the system, OSC, choz's own JACK port (`choz:midi_in`) and choz's own ALSA port (`choz MIDI IN`). Below them are the audio inputs the device or the graph offers, with their level in dB live, grouped by device — and, on PipeWire, the outputs of other applications as well.

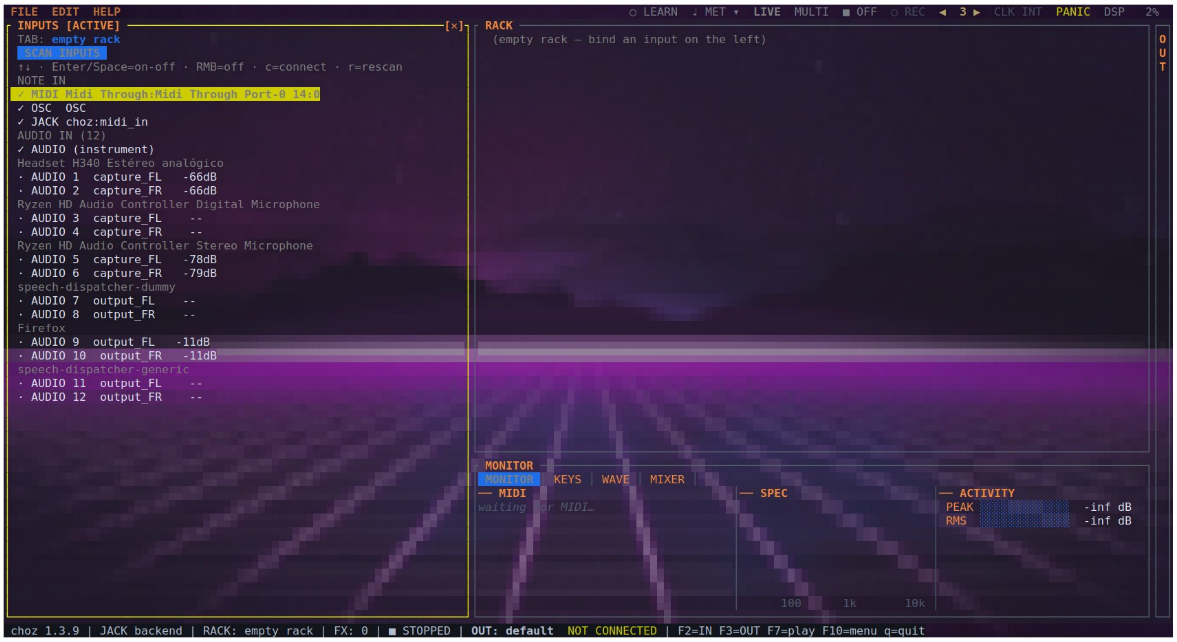


*Figure 4.1 — The IN drawer: note inputs and the audio inputs found, each with its current level.*

| Key | Action |
|---|---|
| Enter / Space | Bind that input to a tab — choz makes one if there is none. Again to unbind (a right-click does the same). |
| `c` | Connect or disconnect the port. |
| `r` | Rescan the inputs. |

choz connects to every enabled MIDI input by itself, so a controller plugged in mid-session starts playing without any wiring. Ports you never want are switched off in this drawer and stay off.

## 4.2 choz MIDI IN

ALSA's sequencer only lets a program *subscribe to* ports that already exist, and a DAW's MIDI output is not one of them: it opens the devices it was told to and publishes nothing anybody else can send to. So choz publishes the port instead. `choz MIDI IN` is a writable port on the ALSA client `choz`: a DAW picks it out of its own MIDI output list and plays a rack tab — with no `snd-virmidi`, no `a2jmidid` and no JACK graph.

- It is opened once and lives as long as choz does. Plugging or unplugging a controller re-scans the inputs, and a port remade there would drop the DAW's subscription every time somebody touched a USB cable — silently, because neither side reports it.
- It is the first row of the note inputs, and it can be switched off like any other port. Switching it off closes it, so it leaves the DAW's list too.
- It carries everything a DIN cable carries: notes, controllers, program change, pitch bend and MIDI Clock — so a DAW's transport can drive choz's tempo (section 8.6).

## 4.3 choz on the JACK/PipeWire graph

With the **JACK** backend (section 13.1), choz registers as the client `choz`:

| Port | What it is |
|---|---|
| `choz:out_1 … out_N` | The output channels: the device's own, plus two more for each tab that has a direct out. |
| `choz:in_1 … in_N` | The capture jacks in the graph, whatever card they belong to. |
| `choz:midi_in` | MIDI from the graph: notes, controllers and clock published by a DAW on the same session. |
| `choz:midi_out` | What the arpeggiator, the sequencer and the tabs send out — to a synth on the graph, or to the DAW's recorder. |

Nothing is auto-connected to the two MIDI ports: what talks to choz is a patch you make, not a guess choz makes for you.

## 4.4 The OUT drawer

**F3** lists the playback device and its channels, and the MIDI output ports available — including `choz:midi_out`.

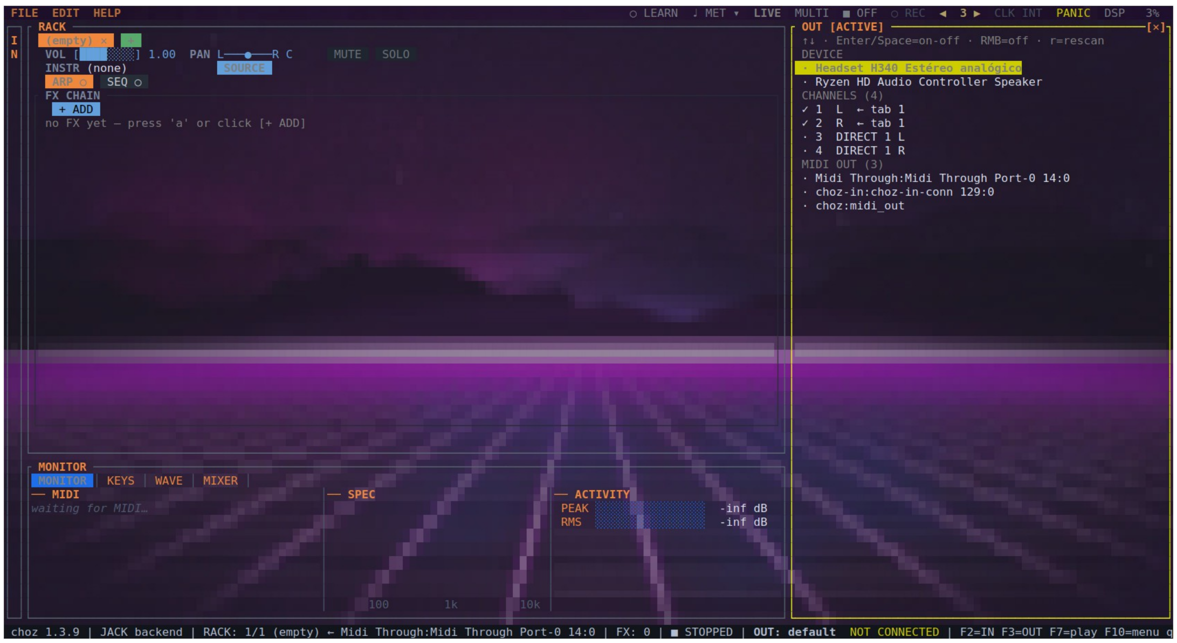


*Figure 4.2 — The OUT drawer: output device, channels, and MIDI output ports.*

## 4.5 Audio inputs and the microphone

An audio input binds to a tab exactly like a note input: open IN, pick the channel, press Enter. The tab gains two controls of its own — **IN**, the input trim (< and >), and **SENS**, the sensitivity of the audio-to-note follower (`;` and `:`) — and the **A→M** button, which turns what comes in into MIDI notes for the instrument or for other tabs.

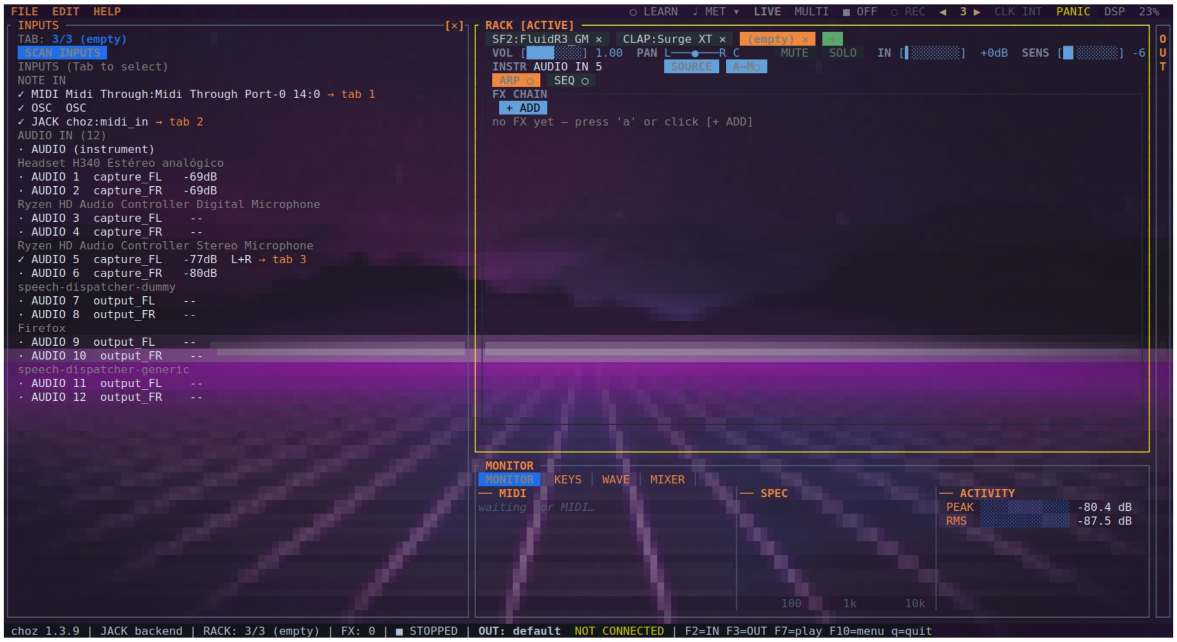


*Figure 4.3 — A stereo microphone bound to tab 3: the IN row with the trim, SENS, and the A→M button.*

- A mono channel stays mono all the way through — one fader on the mixer, one port on the graph — until something in the chain has a reason to pull the two sides apart.
- `n` levels the tab against what it has played since it was loaded; `N` levels the whole rack.
- The feedback guard (section 13.1) cuts the loop when a microphone starts to howl.

## 4.6 Direct outs

See section 11.3: a tab can leave the master mix through a pair of ports of its own, which is how a DAW records each tab on its own track.

# 5. Instruments

## 5.1 The source picker

With a tab active, `1` (or `i`) opens the picker. The tabs across the top filter by format — ALL, CLAP, SF2, WAV, SFZ, LV2, VST2, VST3, DSSI, LADSPA. Type to search, move with the arrows, accept with Enter; **F5** rescans.

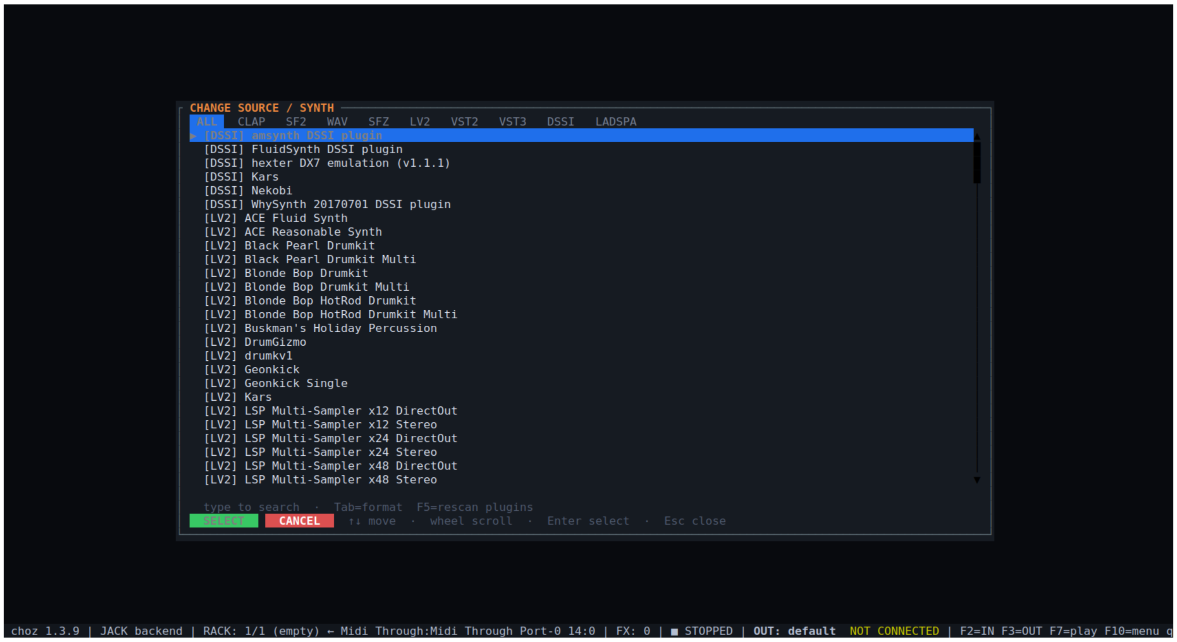


*Figure 5.1 — The source picker: every instrument found, with its format in brackets.*

## 5.2 SoundFonts

choz synthesises SF2 with its own engine, with no dependency on FluidSynth. Loaded, the tab shows the preset on the **BANK** row and the engine's knob box: the bank's reverb and chorus sends, the envelope (Attack, Hold, Decay, Sustain, Release), the filter (Cutoff, Reso), tuning (Coarse, Fine) and output (Pan, Volume).

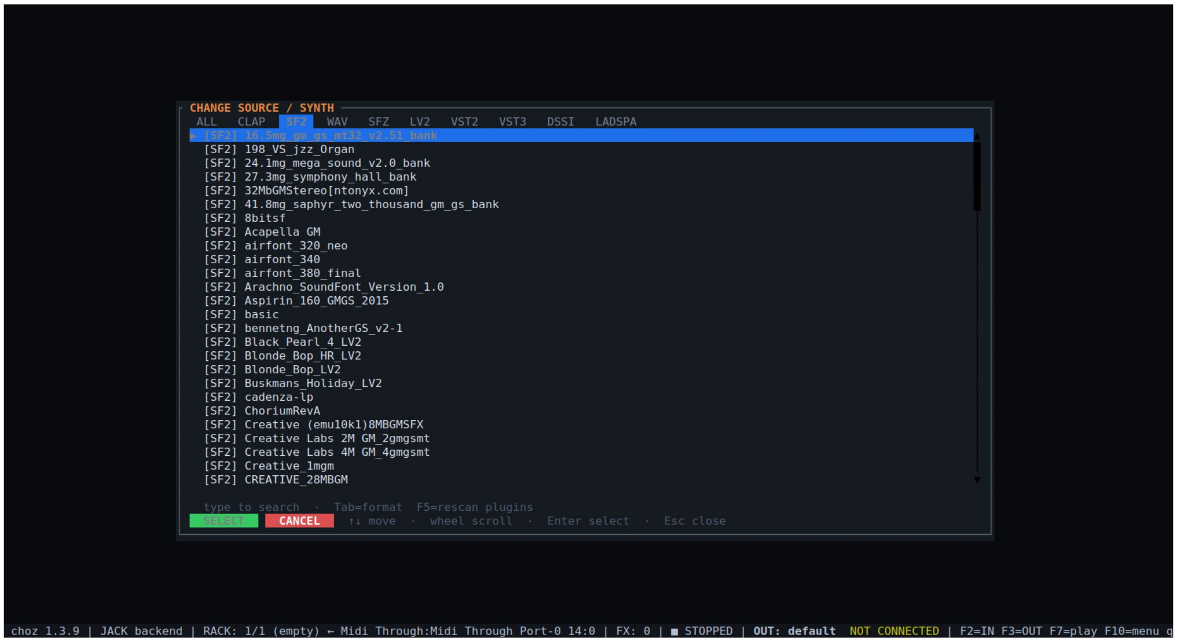


*Figure 5.2 — The picker's SF2 tab: the SoundFont banks installed on this machine.*


*Figure 5.3 — A SoundFont loaded in tab 1: the BANK/PRESET row and the SF2 engine's knob box.*

## 5.3 SFZ and WAV

SFZ instruments play on choz's own 32-voice sampler and a WAV plays as a rack source; both load from the same picker. A SoundFont player installed as a plugin appears under its own format tab and is used like any other plugin.

## 5.4 Banks and presets

`2` (or `b`) opens the bank and preset browser. The highlighted row sounds while you move through it, and **SELECT** keeps it. When a plugin's patches are files rather than numbered programs, the same button opens the categories its own window would show.

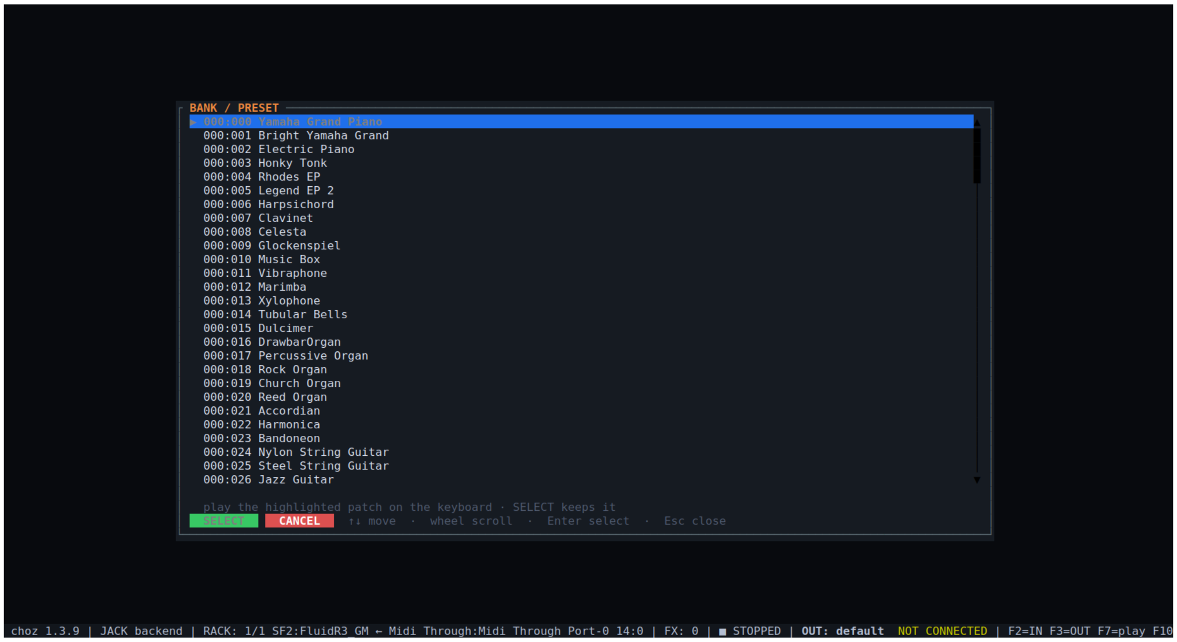


*Figure 5.4 — An SF2 bank browser: the 128 programs of bank 0 and the file's other banks.*

## 5.5 Parameters, pages and search

`p` opens the parameter list with exact values; `l` on a row learns a CC for it. A plugin with hundreds of parameters is shown two rows at a time — **PgUp** and **PgDn** turn the page, and learned CCs move with the box, so eight faders reach every parameter the plugin has. `/` searches a knob by name and jumps to the page holding it.

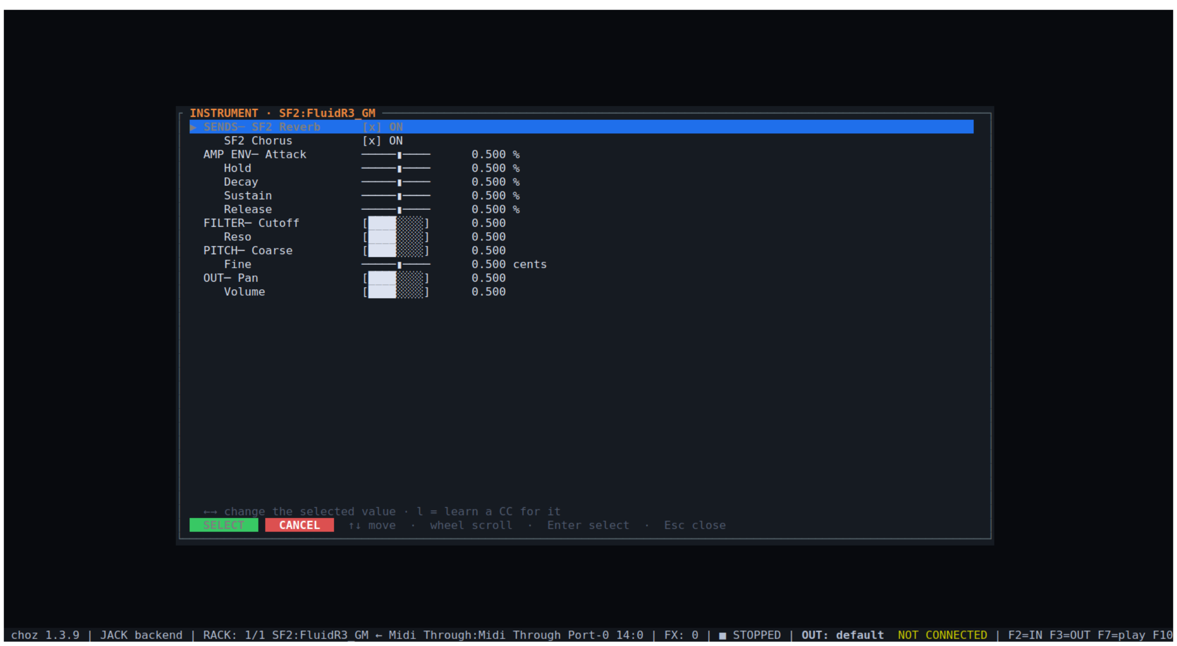


*Figure 5.5 — A SoundFont's parameters: sends, envelope, filter, tuning and output.*

# 6. Effects

## 6.1 The catalogue

a (or **[+ ADD]** in the FX CHAIN section) opens it. The left column is categories with how many effects each holds; the right column is the list. The tabs filter by format, and **BUILT-IN** leaves only choz's own.

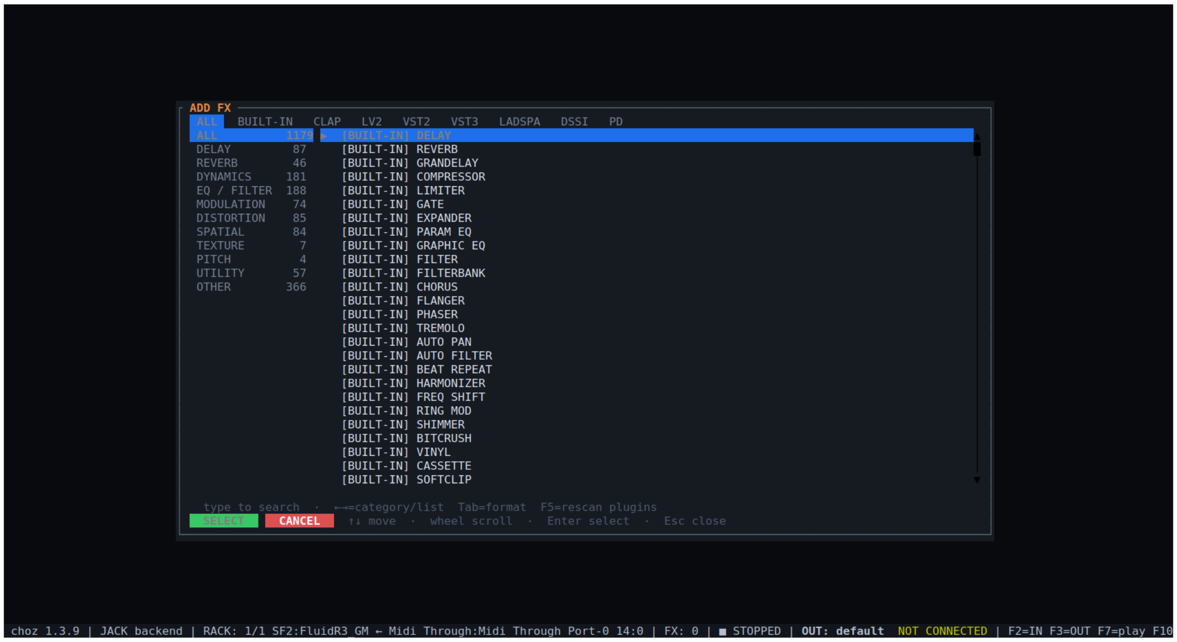


*Figure 6.1 — The effect catalogue: everything available between the built-ins and the installed plugins.*

## 6.2 The chain

A tab holds up to twelve effects, in order. Each shows all of its knobs and, underneath, its slot row: ON/OFF, input and output level in dB, moving it along the chain, deleting it, and its preset buttons.


*Figure 6.2 — The internal reverb in the chain, with its own parameters.*

| **Key** | **Action** |
|---|---|
| Space | Switch the selected effect on or off |
| ← → | Move between effects in the chain |
| ↑ ↓, wheel, w/s | Pick a knob and move it |
| d | Delete the selected effect |
| G | Open the effect's own window, if it is a plugin that has one |
| X | Run the effect isolated in another process |

## 6.3 The built-in effects

Fifty-six effects ship with choz. They need no plugin, they are always present, and they are the ones published in `choz.clap` for other hosts.

| Category | Effects |
|---|---|
| Delay (5) | DELAY, GRANDELAY, SPACE ECHO, REVERSE, MULTI-TAP |
| Reverb (3) | REVERB, SHIMMER, PLATE |
| Dynamics (9) | COMPRESSOR, LIMITER, GATE, EXPANDER, SIDECHAIN, DE-ESSER, TRANSIENT, MULTIBAND, ENVELOPE |
| EQ / Filter (6) | PARAM EQ, GRAPHIC EQ, FILTER, FILTERBANK, ISOLATOR, LADDER |
| Modulation (9) | CHORUS, FLANGER, PHASER, TREMOLO, AUTO PAN, AUTO FILTER, FREQ SHIFT, RING MOD, VIBRATO |
| Distortion (11) | BITCRUSH, VINYL, CASSETTE, SOFTCLIP, SATURATOR, WAVESHAPER, TUBE SAT, AMBER FANG, VELVET FUZZ, EXCITER, BASS ENH |
| Spatial (2) | WIDENER, PAN |
| Texture (4) | BEAT REPEAT, LOOPER, PROTOCOSMOS, Z5 TEXTURE |
| Pitch (4) | HARMONIZER, VOCODER, AUTO-TUNE, PITCH SHIFT |
| Utility (3) | GAIN, PHASE INV, MONO |

## 6.4 Effect presets

Each slot can save and load a preset of its own effect, so a compressor setting that works on a voice can be carried to another tab or another project without saving the whole rack.

## 6.5 Chord source and the gate

- `C` chooses which keyboard the selected effect takes its chord from — the harmoniser, for instance, with two controllers on a hub.
- `c` opens the gate: another tab drives this effect's dry/wet or one of its knobs, with its level or with the notes it receives. That is how a kick ducks a pad, or a vocal opens a delay.

# 7. Plugins

## 7.1 Formats

choz hosts CLAP, LV2, VST2, VST3, LADSPA, DSSI and Pure Data patches, as instruments and as effects. What appears in the rack is what the plugin declares: a parameter that says it cannot be automated is not drawn as a knob, and a parameter whose whole integer range reads as two words is drawn as a switch rather than a fader.

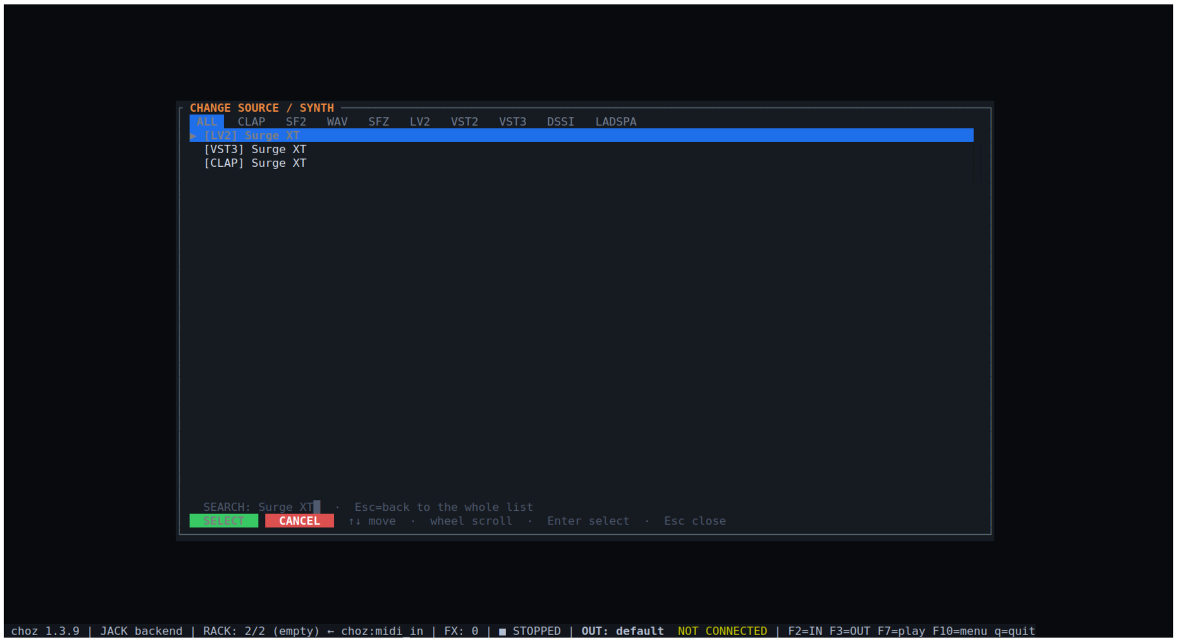


*Figure 7.1 — One synthesiser found in three formats; which you pick only changes the host choz uses.*


*Figure 7.2 — A plugin loaded: its macros, its sends, and the first page of its parameters.*


*Figure 7.3 — Searching a knob by name jumps to the page that holds it and says how many matches there are.*

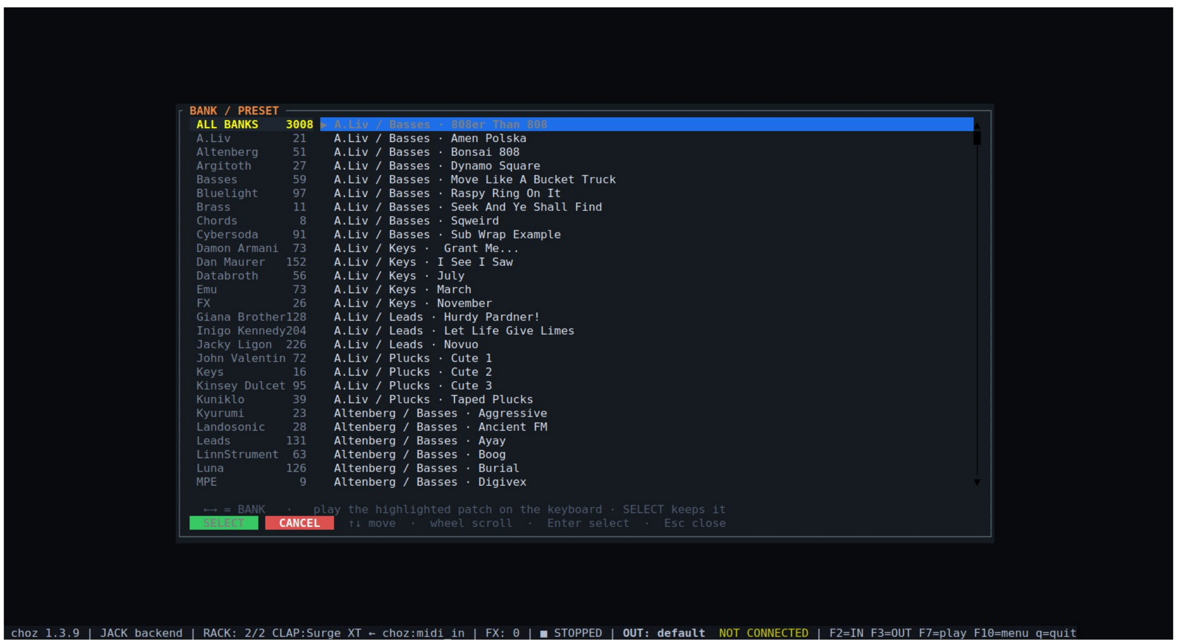


*Figure 7.4 — A plugin whose patches are files: its own categories, browsed from choz.*

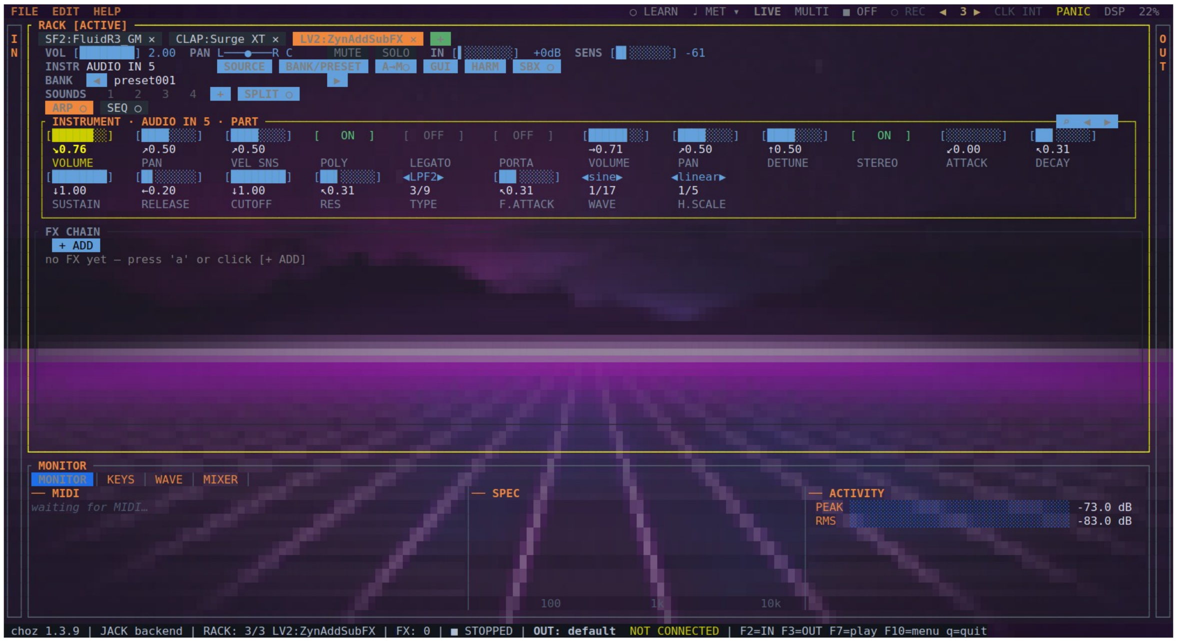


*Figure 7.5 — An LV2 synthesiser in the rack, its parameters as native choz knobs.*

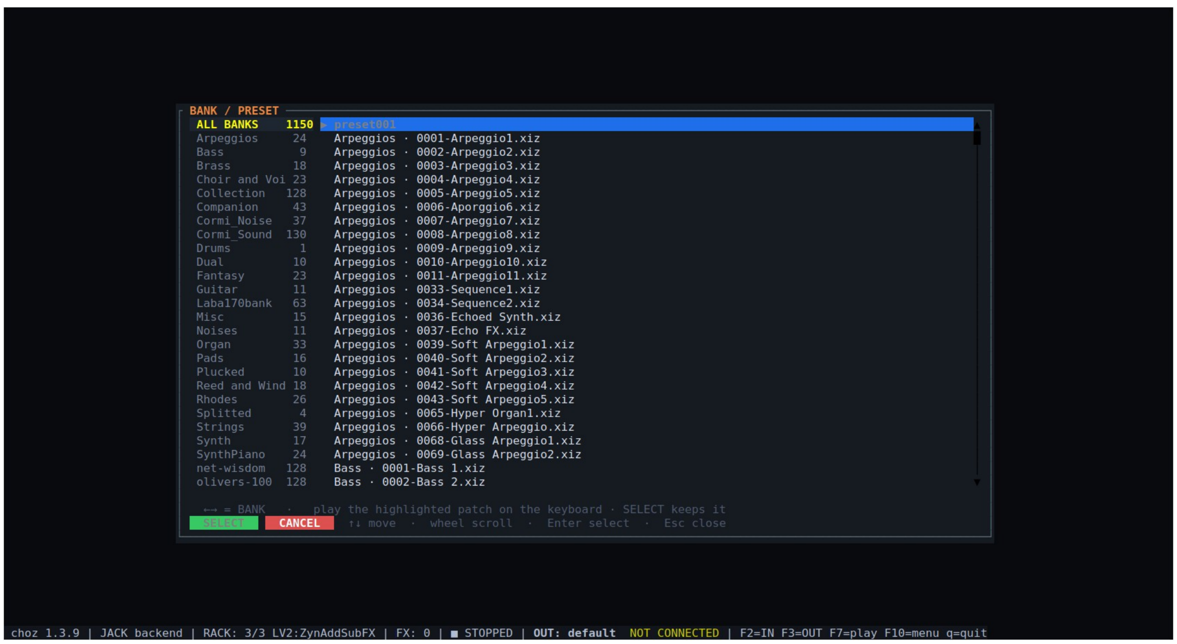


*Figure 7.6 — The same plugin's presets, grouped by folder.*

## 7.2 Scanning and paths

Plugins are scanned in the background and cached, so the picker opens instantly on later starts. EDIT → Rescan plugin paths repeats the scan; **F5** in the picker does the same. Where choz looks is set in Settings (section 13.2).

## 7.3 A plugin's own window

`g` opens the instrument's window and `G` the selected effect's. choz embeds it in an X11 window of its own — LV2 `ui:X11UI`, VST2 `effEditOpen`, CLAP `clap.gui` on the host's timer, VST3 `IPlugView` on the Linux event loop. LADSPA, DSSI and Pure Data have no embeddable window; their parameters are driven from the rack.

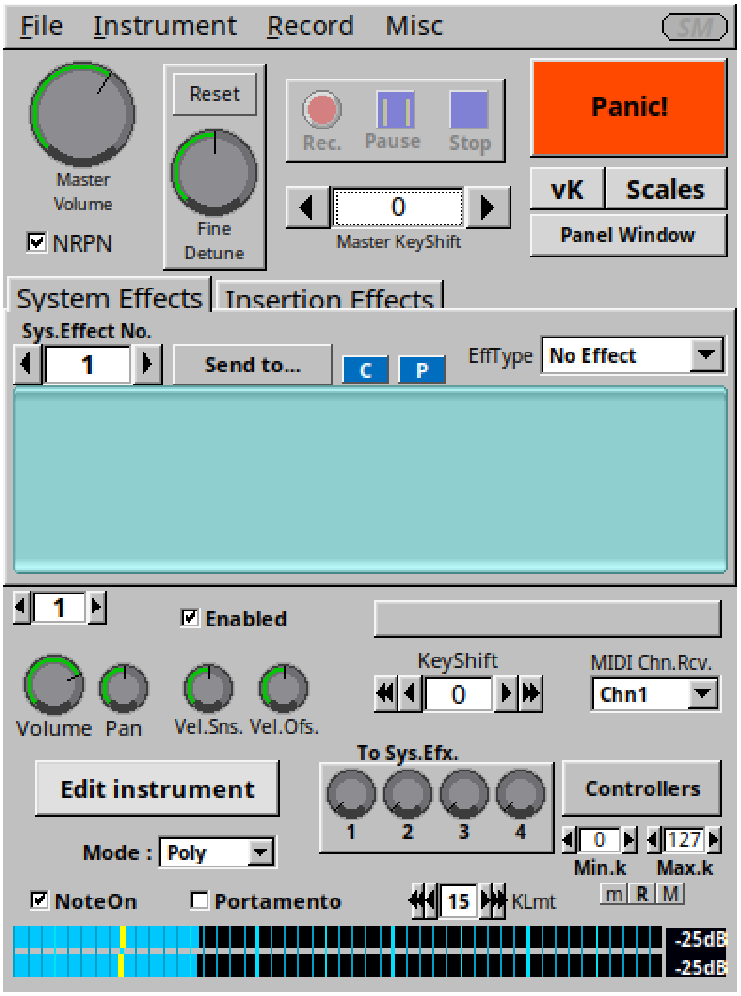


*Figure 7.7 — A plugin's own window, opened from choz with* `g`.

Moves made inside that window are followed by choz, so "move the knob in the plugin, then move a fader on the controller" is enough to bind a CC to it.

An instrument that publishes its oscillator's harmonics — over its own OSC server, say — also gets the **HARMONICS** view (`h`, or the **HARM** button), which draws the harmonic series as bars and edits it in place.

## 7.4 Sandbox and quarantine

`x` runs the instrument in a child process and `X` the selected effect (the **SBX** button does the same). Audio and the window travel over shared memory, so a plugin that crashes does not take choz with it. Before trusting a new plugin, choz probes it in a separate process — a plugin that dies there is remembered and hosted out of process from then on.

**Note.** `CHOZ_SANDBOX_GUI=1` isolates every plugin with a window up front, which is the setting to reach for on a machine where one badly behaved GUI keeps taking the rack down.

## 7.5 Plugin state in a project

A project saves what a parameter list cannot: VST2 chunks, VST3 `IComponent::getState`, `clap.state` and LV2 `state#interface` — the patch chosen in the plugin's own browser comes back with the project.

# 8. Playing and MIDI

## 8.1 LIVE and MULTI

In **LIVE**, every tab plays at once. In **MULTI** (**F4**), each tab listens on a MIDI channel of its own, shown as **CH n** beside the instrument's name — a keyboard split across a rack, or a DAW driving eight tabs down one port.

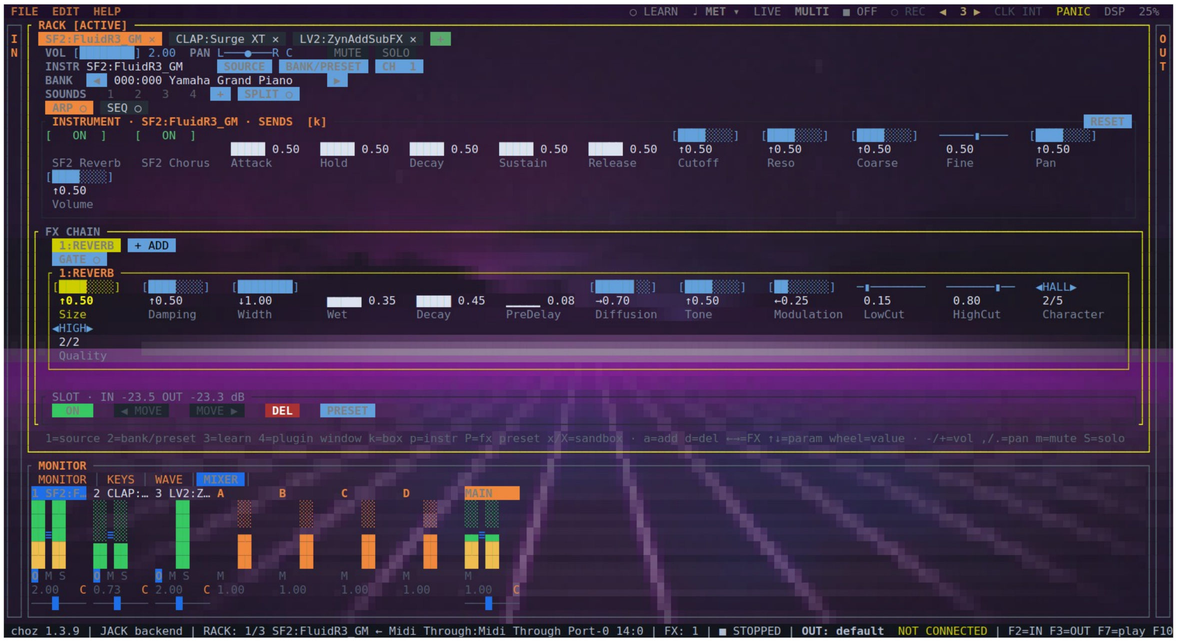


*Figure 8.1 — MULTI mode: each tab bound to its own MIDI channel.*

## 8.2 The monitor

The **MONITOR** panel shows incoming MIDI as it arrives — port, message, note, velocity — beside the spectrum of the output and the peak and RMS meters. It answers three questions at once: did it arrive, what came out, how loud.


*Figure 8.2 — MONITOR: NOTE ON / NOTE OFF with velocity, the spectrum, and the meters.*

## 8.3 MIDI learn

`l` lists the learnable targets of the active tab: volume and pan, mute and solo, bank up and down, parameter page, instrument reset, effects on and moved, add effect, the arpeggiator (on/off, tap, hold), the four sounds, the tabs, and every instrument parameter. Pick one, then move the controller. `3` does it the other way round — pick the control on screen first — and the **LEARN** button lets you learn by clicking a knob.

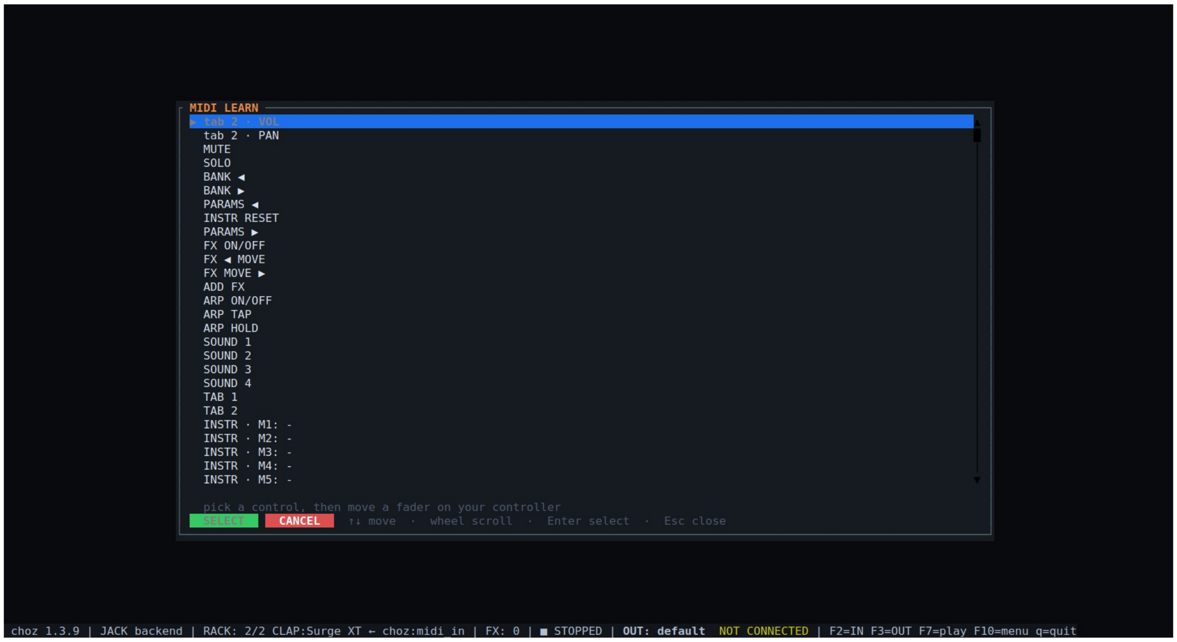


*Figure 8.3 — MIDI learn: the tab's targets, including every plugin parameter.*

## 8.4 KEYS and panic

The **KEYS** panel draws the keyboard with the notes sounding, the pitch bend and the modulation wheel, one colour per tab. `P` (or **F9**, or **PANIC**) silences everything left hanging.

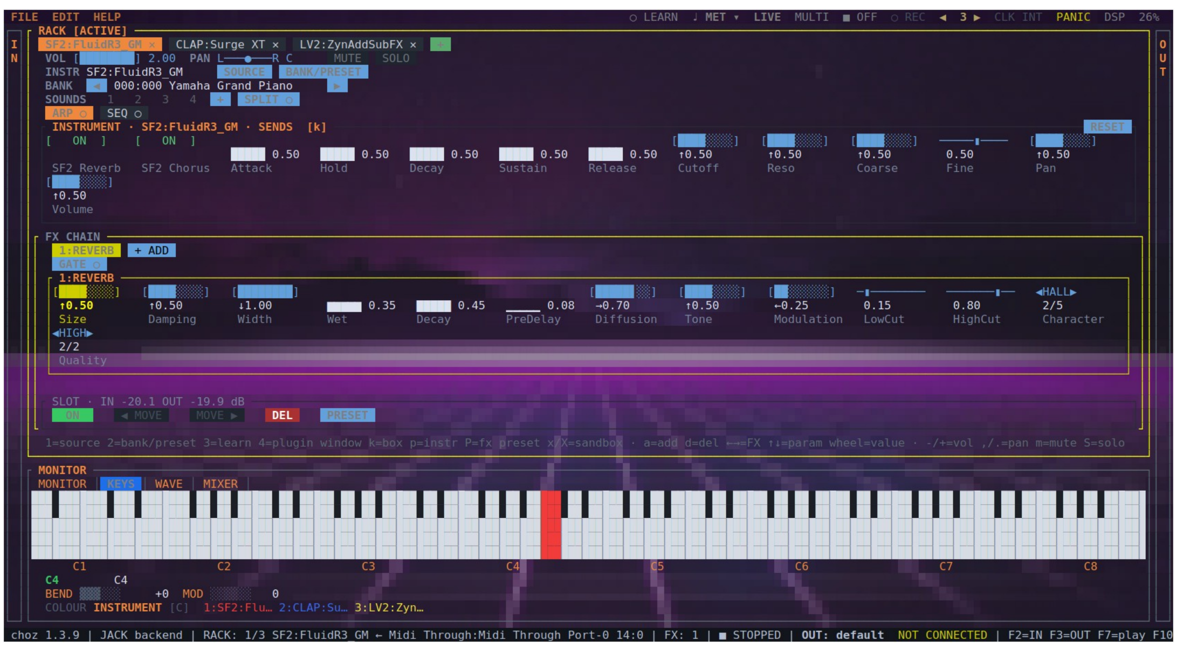


*Figure 8.4 — The KEYS panel: a note sounding, bend and modulation, and each tab's colour.*

## 8.5 OSC

choz listens for OSC as one more note input; `--osc-port` pins the port instead of letting the OS pick one. It appears in the IN drawer and binds to a tab like any other source.

## 8.6 Clock

**CLK** on the menu bar chooses what drives the transport: choz's own clock, or an external device by name. MIDI Clock arriving on a port choz is following moves the tempo, and START, CONTINUE and STOP move the transport with it — so a DAW's play button starts the arpeggiators, the sequencers and the synced delays.

# 9. Arpeggiator and step sequencer

## 9.1 Arpeggiator

A switches on the active tab's arpeggiator (so does the **ARP** button). Its controls are the mode (UP, DOWN and the rest), the division (1/16, 1/8 …), BPM, gate, swing, octaves, **HOLD**, **CHORD** and **TAP**. HOLD behaves like a hardware sequencer's: letting go keeps the chord sounding, and the next key pressed with nothing held starts a new chord instead of adding to the old one.

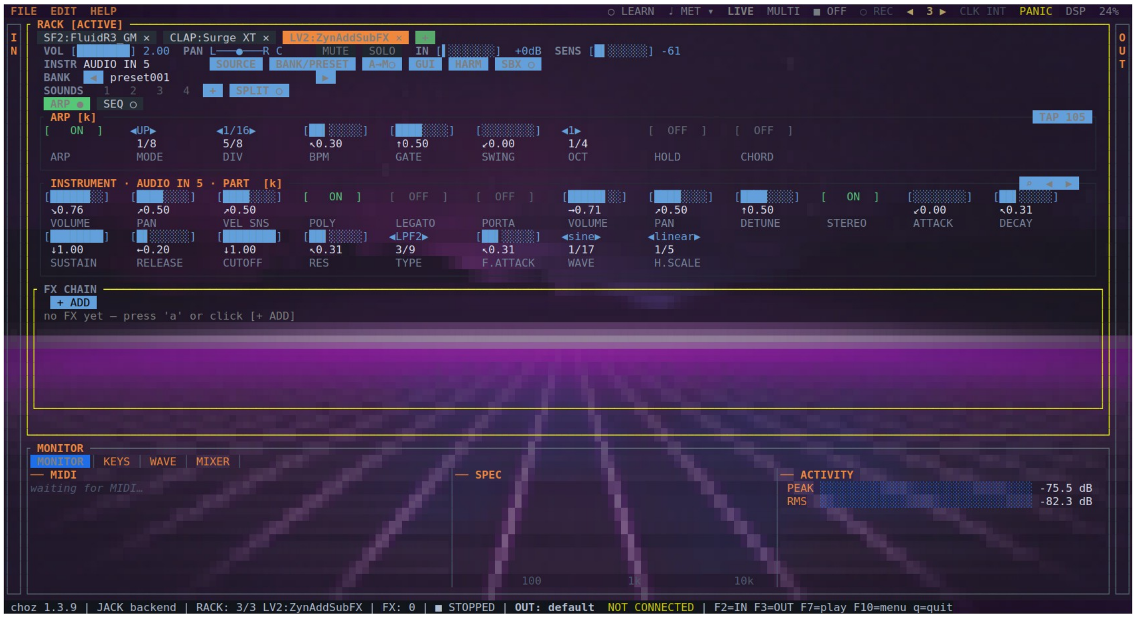


*Figure 9.1 — The tab's arpeggiator: mode, division, BPM, gate, swing, octaves, hold, chord and tap.*

## 9.2 Step sequencer

Every tab also carries a sequencer built like a hardware step machine: eight tracks (A–H), sixteen steps, eight parts and a song chain. It is drawn above the instrument because that is the order the notes travel in. **REC** writes what you play, quantised to the step the play head is on, and each step passes through the tab's arpeggiator when there is one.


*Figure 9.2 — The sequencer: play, record, part, quantise, time signature, song chain, clear, swing, randomness and probability.*

# 10. Transport, tempo and metronome

## 10.1 Transport

**F7** starts and stops the rack, **F8** arms automation recording, **F9** fires panic — the same three controls that sit on the menu bar. The transport is what every sequencer, every synced delay and the automation count against.

## 10.2 Metronome

**F6** switches the click on; the ▾ beside **MET** opens its options: click, tempo, beats and unit of the bar, grouping, sound, output and level. It keeps counting with the transport stopped — which is usually when it is needed — and it counts the same tempo the tempo-synced plugins read.

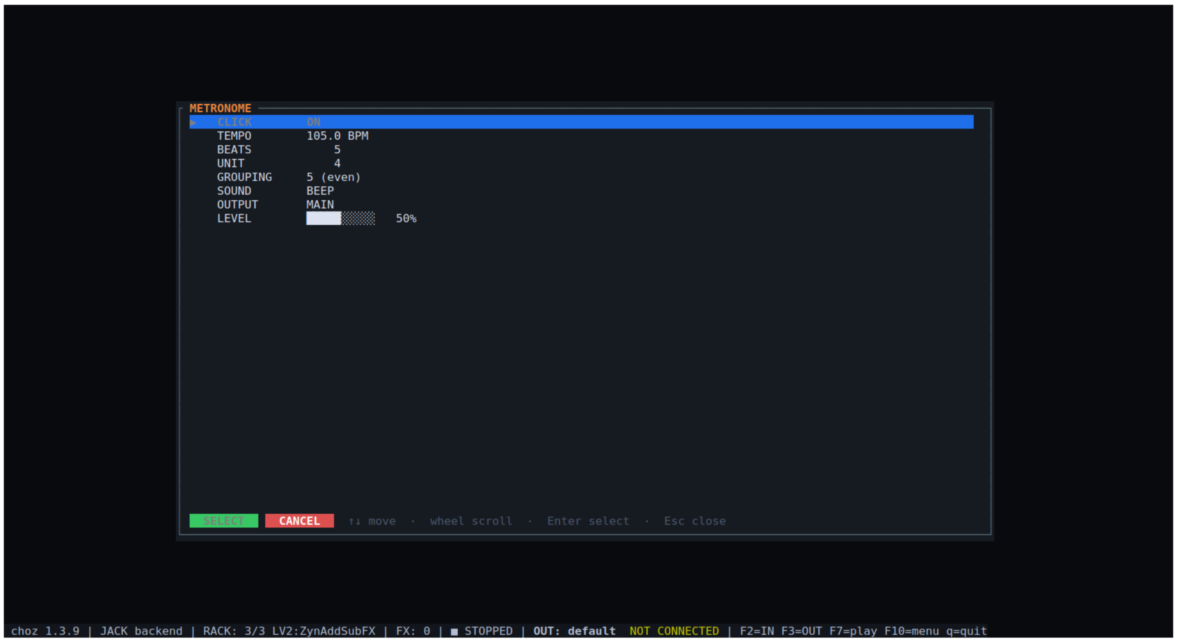


*Figure 10.1 — The metronome's options: click, tempo, time signature, grouping, sound, output and level.*

## 10.3 TAP

**TAP** sits immediately left of **MET**: four clicks set the transport's tempo. It is the same count the arpeggiator's own TAP does — the average of the last four intervals, with a gap longer than two seconds starting a new count, because that was somebody stopping rather than playing very slowly.

- The button **lights on every tap**: playing a tempo in is a gesture with no other feedback, and a button that never moves cannot say whether the click landed.
- On the **fourth tap the metronome starts by itself** — counting four in is counting for a click, and the click you were counting for is the one you want to hear.
- Taps are quarter notes whatever division the arpeggiator is running, and the tempo they set is the transport's, so everything synced follows.

**Tip.** TAP is also a MIDI learn target, so a pad on a controller can play the tempo in without reaching for the mouse.

# 11. Mixer, routing and views

## 11.1 The mixer

The **MIXER** panel shows the whole rack at once as channel strips: one vertical fader per output channel with a link between the two, pan, and the **O M S** row under the fader (output, mute, solo).

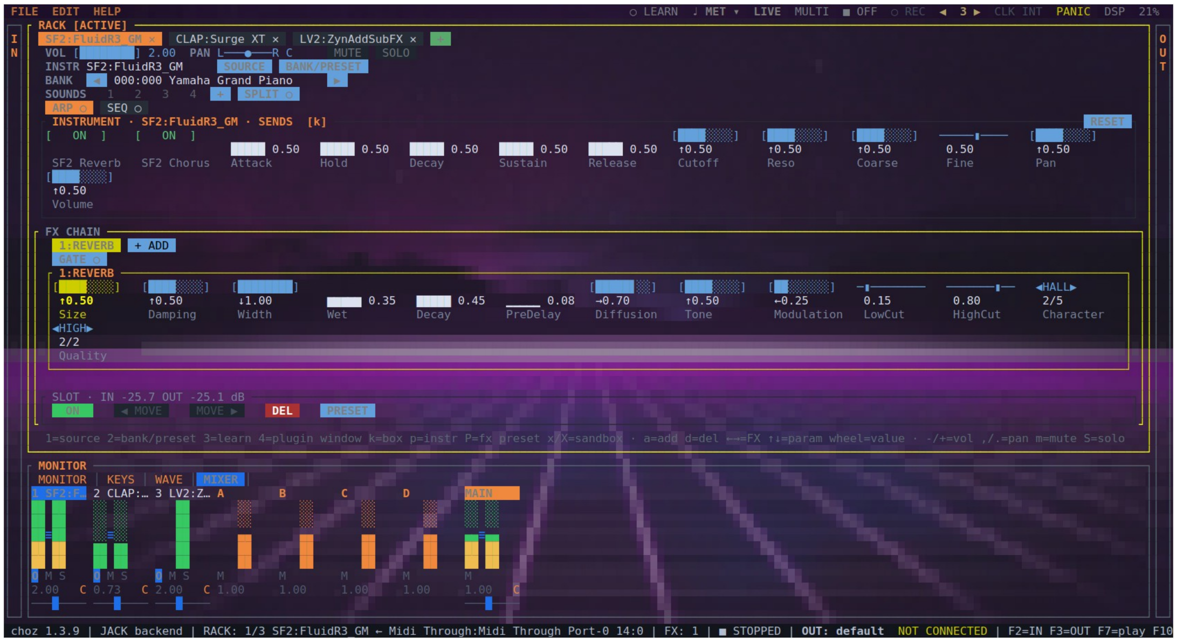


*Figure 11.1 — MIXER: tabs, groups and the main mix, each strip with its faders, pan and O M S.*

| Key | Action |
|---|---|
| Tab | Focus the mixer while it is on screen (clicking a strip does the same) |
| ↑ ↓ / wheel | The strip's level, in the same step VOL uses in the rack |
| ← → | Next strip: the tabs, then the four groups, then the main mix |
| `l` / `k` | Link or unlink the strip's two channels / choose which side the arrows move |
| `,` `.` / `m` / `S` | Pan / mute / solo |

## 11.2 Groups and the main mix

Four groups (A, B, C, D) sit between the tabs and the main mix, for a submix with one fader on it. The main is a stereo pair with its two sides tied by default and a balance rather than a pan: a centred main is untouched, and the sides are broken apart only to trim one wedge against the other.

## 11.3 Direct outs

The **O** button on a strip opens the list of where that tab sums: **MAIN**, the four groups, and **DIRECT n** — a pair of JACK ports of the tab's own with nothing on the other end, for a DAW to patch into. The main fader only ever touches the first pair, so a tab on its direct out is off the master mix by construction.

- **One direct per tab, at a fixed place.** Tab 1 owns the first pair, tab 2 the second, and each strip offers only its own — a shared list let two tabs pick the same one and land on the same ports. The place never moves: a layout that shifted when the rack changed shape would break the patch in the DAW that the feature exists for.
- **As wide as the tab is.** A mono tab routes to a single port — what a DAW wants to record off a guitar is one track, not two identical ones. It becomes stereo the moment something in the chain has a reason for the sides to differ: a reverb tail, a chorus spread, a pan, a ping-pong delay, or any hosted plugin. A bypassed effect does not count, and the width is re-measured when the chain or the jack changes.
- The strip's destination cell says where it is going: `D1…D8` for a direct out, the group's letter for a group, nothing for the main mix.

## 11.4 The views

| View | What it answers |
|---|---|
| MONITOR | Did the note arrive, what came out, how loud: the MIDI log, the spectrum and the meters, side by side. |
| KEYS | What is sounding right now — held notes, bend and modulation, one colour per tab. |
| WAVE | The waveform of what is playing. |
| MIXER | The whole desk: every tab, the four groups and the main mix. |
| HARMONICS | Not a panel tab but a view of its own (`h`): the harmonic series of an instrument that publishes it, edited in place. |

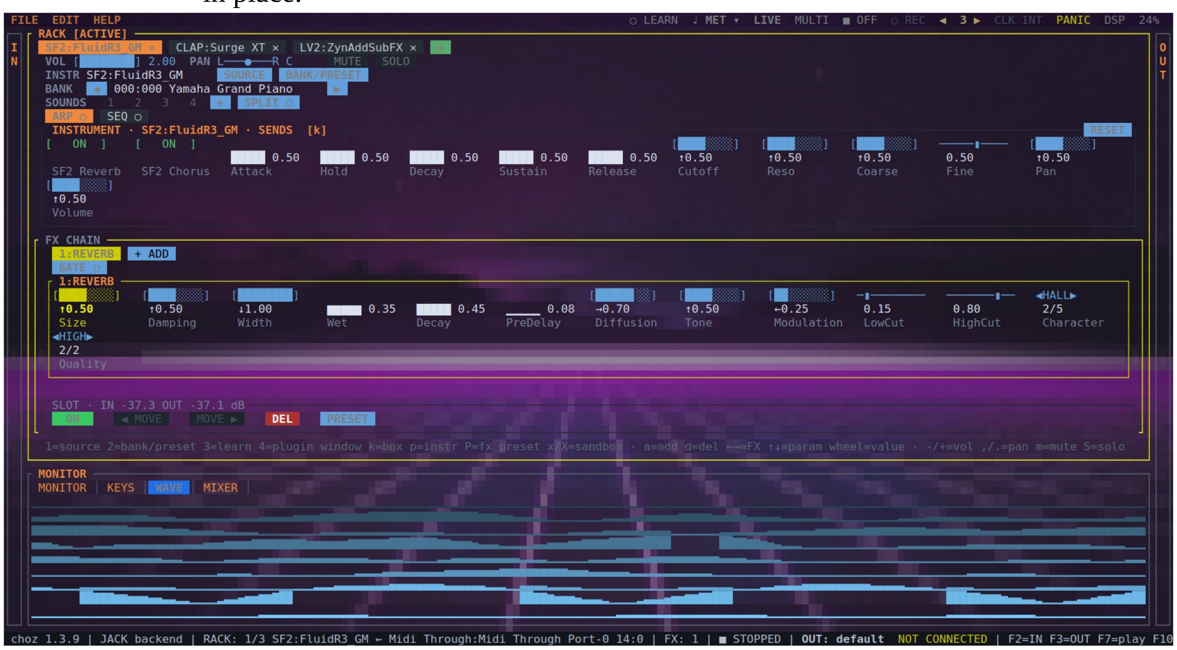


*Figure 11.2 — The WAVE panel with a SoundFont's signal going through the internal reverb.*

# 12. Projects

**F10** opens the menu bar. **FILE** makes, saves and opens projects; **EDIT** leads to Settings and to rescanning the plugin paths; **HELP** shows the About box.

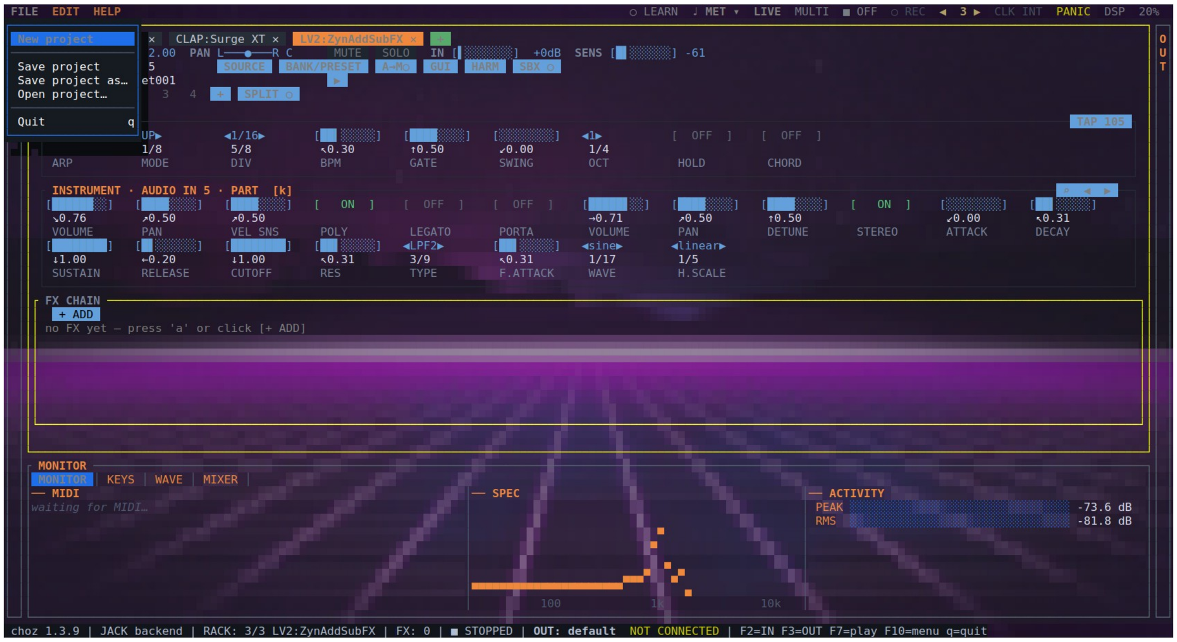


*Figure 12.1 — The FILE menu: new project, save, save as, open and quit.*

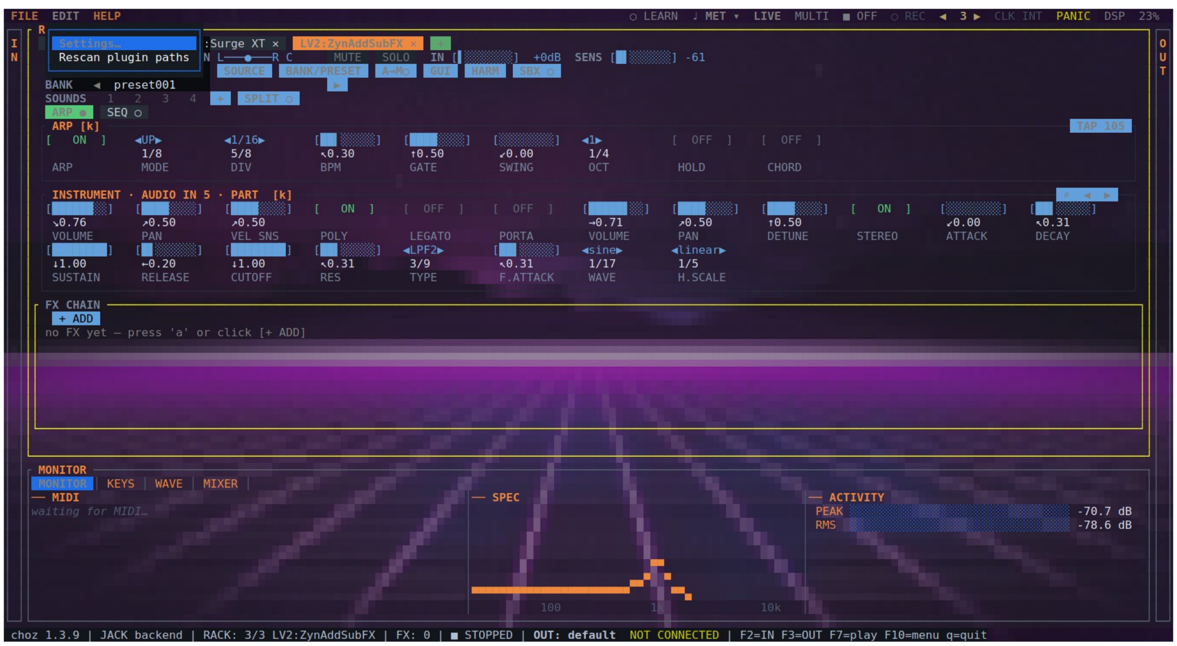


*Figure 12.2 — The EDIT menu: Settings and rescanning the plugin paths.*

A project is readable YAML that can be kept under version control. Besides the rack and its routing it carries each plugin's own state (section 7.5). Looper takes are saved as WAV inside `<project>.loops/` next to the file: moving a project means moving the `.yml` and its folder. A take recorded at another sample rate is resampled on open, keeping its pitch and its length in seconds.

# 13. Preferences

EDIT → Settings has three tabs: **AUDIO**, **THEME** and **LANGUAGE**.

## 13.1 Audio

The engine: backend (AUTO, ALSA or JACK), device, input, sample rate, buffer size, tempo, time signature, feedback guard, SF2 engine, the resulting latency and the PipeWire quantum.

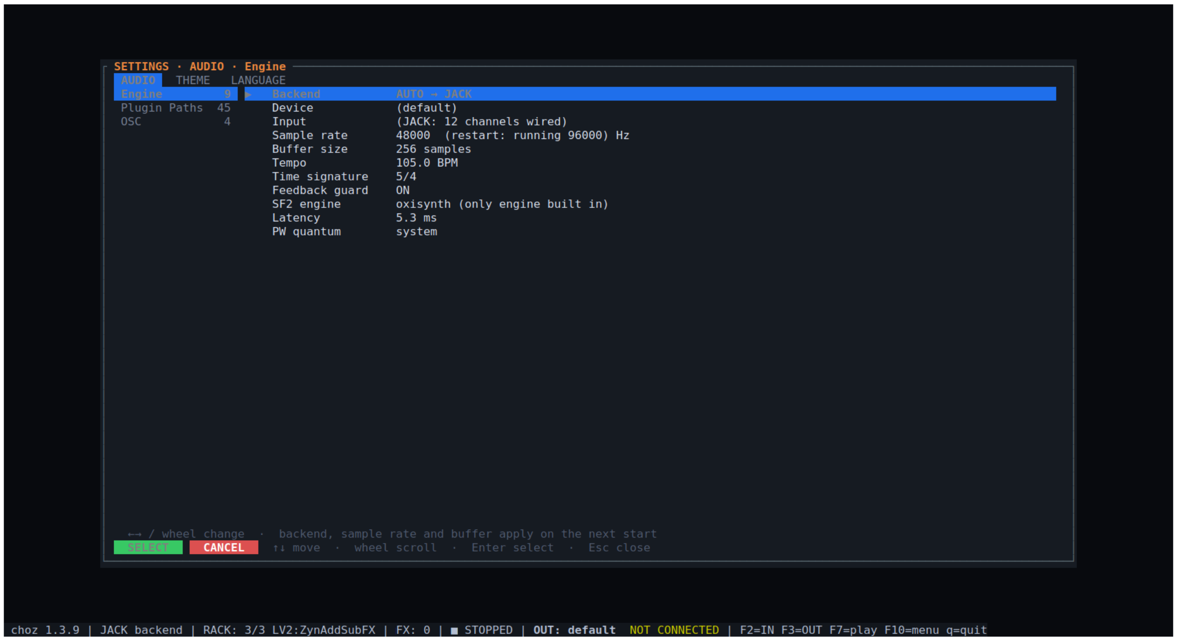


*Figure 13.1 — Settings · AUDIO: backend, device, rate, buffer and the latency they add up to.*

**Note.** The backend, the sample rate and the buffer size take effect on the next start. Choosing **JACK** is what puts `choz:out_*`, `choz:in_*`, `choz:midi_in` and `choz:midi_out` on the graph.

## 13.2 Plugin paths

Per format, every search folder with how many plugins were found in it, or `(missing)`. The buttons edit, add, browse, remove or restore the defaults. The format's environment variable, when set, replaces the list (section 15).

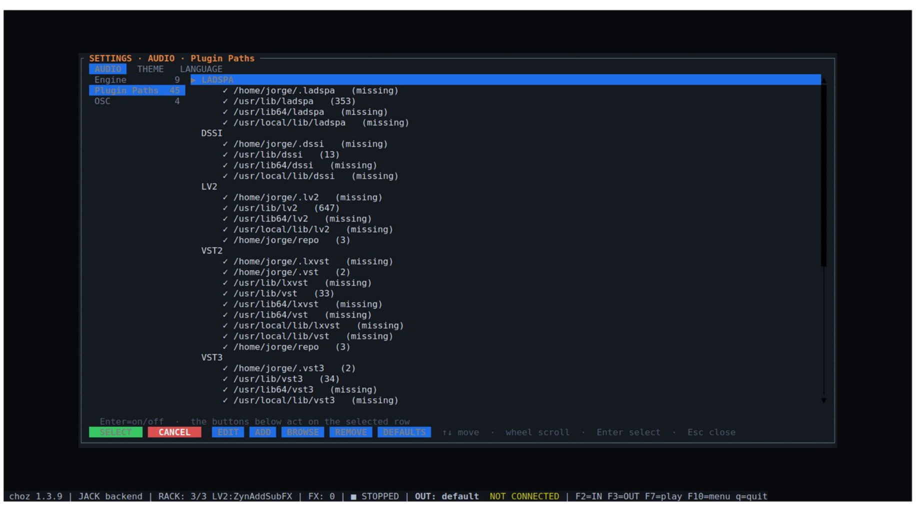


*Figure 13.2 — The search paths per format, with how many plugins each holds.*

## 13.3 Theme and wallpaper

Every change here applies live and is saved.

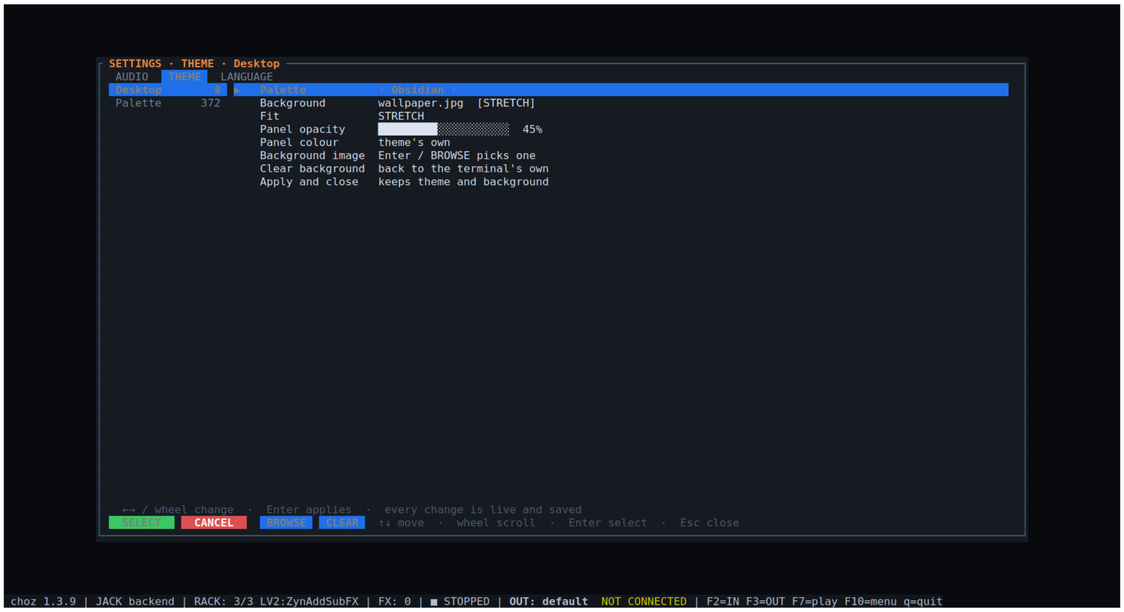


*Figure 13.3 — Settings · THEME: palette, wallpaper, fit and panel opacity.*

| Row | What it does |
|---|---|
| Palette | 372 colour schemes — choz's own eleven first, then the Gogh terminal schemes. ← → try one in place without leaving the row; Enter opens the full list at the scheme currently worn. |
| Background | Cycles the terminal's own background and the scheme's flat colour. |
| Pick image | Opens the file browser on the project's `assets/` to set a wallpaper; the installed wallpapers live there. |
| Fit | Stretch or tile. Present only while an image is set. |
| Panel opacity | How strongly the panel colour washes over the wallpaper. Absent on the terminal's own background: choz cannot read that colour, so there is nothing to be translucent against. |
| Panel colour | Which colour that wash is — the scheme's own, or any palette entry. |
| Clear background | Back to the terminal's own background. |

On kitty, Ghostty and WezTerm the wallpaper is drawn with the terminal's graphics protocol; elsewhere it is painted with cell colours, and `CHOZ_KITTY_BG=0` forces that path. The picture is re-sent only once the window size has settled, so dragging a window edge does not re-scale a large image on every frame.

## 13.4 Language

The **LANGUAGE** tab switches the interface language live, with no restart. Nine are available:

| Language | Code |
|---|---|
| English | en |
| Español | es |
| Português | pt |
| Français | fr |
| Italiano | it |
| Deutsch | de |
| Русский | ru |
| 日本語 | ja |
| 中文 | zh |

At first start choz takes the language from the environment (`LC_ALL`, then `LC_MESSAGES`, then `LANG`) and falls back to English. What you pick here is saved in `ui.json` and wins from then on.

# 14. Reference: keyboard shortcuts

| **Key** | **Where** | **Action** |
|---|---|---|
| F2 / F3 | anywhere | The IN and OUT drawers |
| Enter / Space | a drawer row | Bind that input to the active tab (or unbind it) |
| c / r | IN drawer | Connect or disconnect the port / rescan |
| F4 | anywhere | LIVE ↔ MULTI |
| F5 | anywhere | Bottom panel: MONITOR / KEYS / WAVE / MIXER |
| F6 | anywhere | Metronome (the ▾ opens its options) |
| F7 / F8 / F9 | anywhere | Start the rack / arm automation / panic |
| F10 | anywhere | The menu bar |
| [ / ] | rack | Previous / next tab |
| 1 or i | rack | Change the tab's instrument |
| 2 or b | rack | Bank and preset |
| p | rack | The instrument's parameter list |
| / | rack (instrument) | Search a knob by name |
| PgUp / PgDn | rack | Page the knob box (learned CCs move with it) |
| h | rack | HARMONICS, for an instrument that publishes them |
| k | rack | Move the cursor between instrument and effects |
| a / d | rack | Add an effect / delete the selected one |
| Space | rack (FX) | Switch the effect on or off |
| g / G | rack | The instrument's own window / the selected effect's |
| x / X | rack | Run the instrument / the effect isolated |
| l | rack | MIDI learn |
| 3 | rack | MIDI learn, picking the control first |
| A | anywhere | The active tab's arpeggiator |
| C / c | rack (FX) | The chord's keyboard / the gate from another tab |
| v | rack | Split: which sound each octave plays |
| < > and ; : | rack | Input trim and A→M sensitivity on an audio tab |
| **-** / + | rack | The tab's volume |
| , / . | rack, MIXER | Pan |
| m / S | rack, MIXER | Mute / solo |
| n / N | rack, MIXER | Level the tab / the whole rack |
| ↑ ↓ / wheel | MIXER | The strip's level |
| ← → | MIXER | Next strip: tabs, groups, main |
| Tab | MIXER | Focus the mixer |
| P | anywhere | Panic: silence every note |
| q | anywhere | Quit |

TAP, LEARN, the **O** button on a strip and the metronome's ▾ are mouse controls.

# 15. Reference: environment variables and files

| Variable | Effect |
|---|---|
| PIPEWIRE_LATENCY / PIPEWIRE_QUANTUM | Set by choz from the configured buffer before it opens the JACK client |
| CHOZ_SANDBOX_GUI=1 | Isolate every plugin with a window, not only the ones that have crashed before |
| CHOZ_PROBE_RUNS=N | How many times a plugin is probed before it is called safe (3 by default) |
| CHOZ_KITTY_BG=0 | Draw the wallpaper with cell colours instead of the terminal's graphics protocol |
| CHOZ_CLAP_STRICT_TEARDOWN / CHOZ_LV2_STRICT_TEARDOWN | Destroy plugins properly instead of leaking the ones known to crash on teardown (debugging) |
| LC_ALL / LC_MESSAGES / LANG | The interface language at first start, until one is chosen in Settings |
| LV2_PATH, VST_PATH, VST3_PATH, CLAP_PATH, LADSPA_PATH, DSSI_PATH, SF2_PATH, SFZ_PATH | Replace the search path for that format |

| File in `~/.local/state/choz/` | What it holds |
|---|---|
| `choz.log` | The log |
| `plugins.json` | The scan cache: what was found, where, and when |
| `plugin-paths.json` | The search folders per format |
| `plugin-verdicts.json` | Which plugin passed quarantine and which did not |
| `plugin-sandbox.json` | Which plugins are hosted out of process |
| `ui.json` | Interface preferences: theme, wallpaper, language, panel opacity |

# 16. Troubleshooting

| Symptom | What it usually is |
|---|---|
| No sound, and MONITOR shows no MIDI | The port is not bound. Open IN (F2), select the port, press Enter — the row should say → **tab n**. If the port is not listed at all, press `r` to rescan. |
| MONITOR shows MIDI but nothing sounds | The tab has no instrument (press `1`), the tab is muted or soloed away in the MIXER, or in MULTI the tab is listening on another channel (F4). |
| A DAW cannot see choz | For MIDI, look for `choz MIDI IN` in the DAW's MIDI *output* list — it is a destination, not a source. If it is missing, it was switched off in the IN drawer. For audio, the backend must be JACK (Settings → AUDIO) and choz must be restarted after changing it. |
| Clicks and dropouts | Raise the buffer size in Settings → AUDIO, or lower the DSP load by bypassing effects. The DSP figure on the menu bar says which of the two you are short of. |
| Dropouts, but DSP is low | The graph itself is dropping cycles, and the cause is outside choz: another client, or another card xrunning in the same graph. choz says so in the log when it detects it; `journalctl --user -q \| grep spa.alsa` names the device. |
| Everything is slow when built from source | A debug build renders roughly ten times dearer than a release one. Build with `--release`. |
| choz disappeared when a plugin crashed | Run that plugin isolated: `x` for an instrument, `X` for an effect, or `CHOZ_SANDBOX_GUI=1` for every plugin with a window. A plugin that has crashed once is remembered and hosted out of process from then on. |
| A plugin is not in the picker | Its folder is not on the search path: Settings → Plugin Paths, then add it and rescan (EDIT → Rescan plugin paths, or F5 in the picker). A format's environment variable, when set, replaces the whole list for that format. |
| A plugin window does not open | LADSPA, DSSI and Pure Data have no embeddable window by design — use the knobs in the rack. Otherwise choz needs X11 or XWayland. |
| A microphone tab howls | The feedback guard cuts the loop; leave it on (Settings → AUDIO). Lower **IN** on the tab and check that the tab is not routed into what the microphone hears. |
| The wallpaper is missing or flickers | Outside kitty, Ghostty and WezTerm it is painted with cell colours. `CHOZ_KITTY_BG=0` forces that path everywhere; Clear background turns it off. |
| The tempo drifts or will not change | Something else is driving the clock: check **CLK** on the menu bar. On an external source, the tempo is the sender's and TAP will not hold. |

# 17. Glossary

| **Term** | **Meaning in this guide** |
|---|---|
| Rack | The middle of the screen: all the tabs, and what they hold. |
| Tab | One channel of the rack: an input, an instrument, an effect chain, an arpeggiator, a sequencer, a fader and a destination. |
| Drawer | The IN and OUT lists at the screen's edges (F2, F3). |
| Direct out | A pair of graph ports belonging to one tab, off the master mix, for a DAW to record. |
| Group | One of four submixes (A–D) between the tabs and the main mix. |
| Sandbox | Running a plugin in a child process so its crash cannot take choz down. |
| Quarantine | The probe choz runs on an unknown plugin before hosting it in its own process. |
| Gate (on an effect) | Another tab driving this effect's dry/wet or a knob, with its level or its notes. |
| A→M | Audio to MIDI: turning what an audio tab hears into notes. |
| Quantum | PipeWire's block size, which choz asks for from the configured buffer. |
| xrun | A cycle the audio graph failed to deliver on time; heard as a click. |

**Notes on this edition.** The screenshots come from choz 1.3.9 running with the plugins installed on that machine; the screen is unchanged in 1.3.10 apart from the TAP button described in section 10.3, which sits immediately left of MET. Plugin names, banks and presets shown are examples, not requirements.